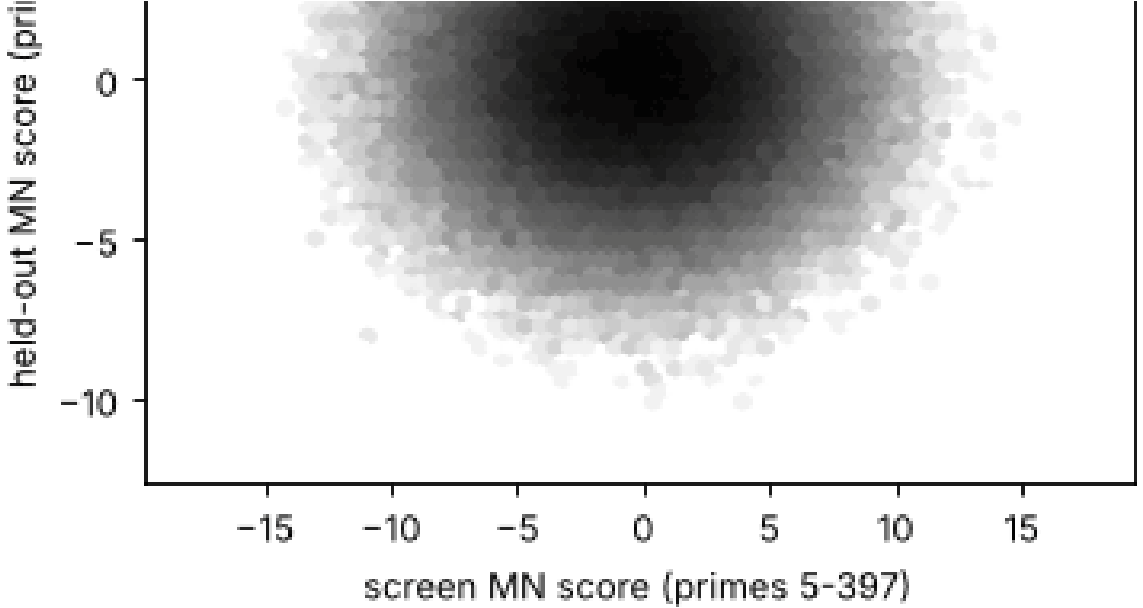
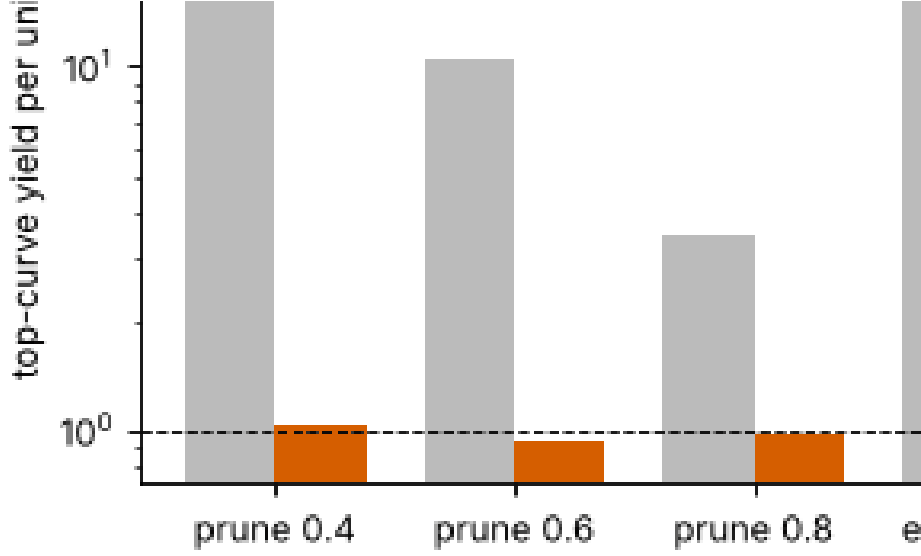

held-out MN score (pri
0
−5
−10
−15
−10
−5
0
5
10
15
screen MN score (primes 5-397)


top-curve yield per uni
10^1
10^0
prune 0.4
prune 0.6
prune 0.8
e

Held-out-prime control for a Mestre–Nagao rank sieve

Status: **exploratory (retrospective)**. The protocol and the result are published together. This is side work: high-rank curves over Q are not an index-calculus improvement, and the xedni lift that would connect them to ECDLP is known to fail (Jacobson–Koblitz–Silverman–Stein–Teske 2000). No leaderboard, benchmark or IC row changes.

Question

A report proposed three speed-ups for a Mestre–Nagao (MN) screen of E_t: $y^2 = x^3 + (t^2 - 3)x + (t^3 + 5)$:

1. per-prime lookup tables,
2. early abort after the first 25 primes,
3. residue-class pruning mod 5–23,

and claimed that pruning gives "about 10× more top curves per unit of compute". Do (2) and (3) enrich anything beyond the score they filter on?

Method

- Screen score: $\Sigma\ -a_l(t) \log l / l$ over the 76 primes 5–397, with bad reduction contributing 0. "Top curves" are the top 10^{-5} of $t = 1 \ldots 10^7$ (100 curves).
- **Held-out control:** the same sum over the primes 401–1193, which are disjoint from the screen. A curve with elevated rank has elevated MN sums at every prime, so real enrichment must survive on held-out primes.
- **Positive control:** $y^2 = x^3 + (t^2 - 3)x + t^2$ has the point $(0, t)$, so its generic rank is at least 1. Its held-out mean must exceed the test family's, which shows the instrument can see rank.
- **Correctness:** table scores match a naive Legendre-symbol computation for $t = 1 \ldots 300$ (max $|\Delta| = 3.7e\text{-}6$, float32).

Reproduce: `python3 run.py && python3 plot.py`. Single CPU core, Intel Xeon @ 2.10 GHz, numpy 2.5.3, Python 3.13. Full numbers and source hash are in `result.json`.

Results

Arm	Enrichment on screen score	Enrichment on held-out primes (top 0.1%)
Prune, keep 0.4 (0.16% of space)	36.9×	1.05×
Prune, keep 0.6 (2.8% of space)	10.4× (recall 29%)	0.93×
Prune, keep 0.8 (22% of space)	3.5× (recall 76%)	0.98×
Early abort (25 primes, keep 1%)	recall 98%	0.91×

- Correlation between screen and held-out scores: 2.6e-6.
- Held-out mean, all curves: 0.217 (sd 2.64). Held-out mean, top 100 by screen: 0.112 ± 0.288 (SE). The top screen curves show no rank signal.
- Positive control ($t \le 10^6$): the test family's held-out mean is 0.217 ± 0.003; the rank ≥ 1 family's is 1.137 ± 0.003. The instrument resolves one unit of rank at about 350 SE.
- Speed: naive 4,812 µs per curve, table gather (no tiling) 799 ns per curve. Timings vary by ~5% between reruns on this shared VM. The report's tiled 103 ns was not reproduced; that layout was not built here.

Interpretation

- The pruning claim reproduces exactly on its own metric: 10.4× at keep 0.6. That gain is circular, because each MN term depends only on t mod l, so keeping favourable residues raises the score by construction. It gives no enrichment on held-out primes, where the instrument provably detects rank.
- Early abort is safe as a cost cut, but its recall only measures how well a partial score predicts the full score. It is not evidence about rank.
- In this toy family, the top MN-scoring curves are noise at small primes. The family can't test a sieve's ability to find rank.

Limits

- Held-out MN is a proxy, not rank. A stronger check would compute analytic rank or run a 2-Selmer computation on the top curves.
- One toy family. Pruning could still help in a family whose residue structure reflects real rational points; the held-out test is the way to show it.
- The positive control is family-level, not curve-level.
- Timings come from a shared cloud VM, with no isolation gate or repeats. They are indicative only.
- The claim that Elkies's 2006 family was re-screened in 2019 was not checked.

Files

`run.py` (experiment), `plot.py` (figure and PDF), `result.json`, `figure.svg`, `figure.png`, `report.pdf`.